



Autonomous Drone Detection & Response System

Project Engineering

Year 4

Aidan Malone

Bachelor of Engineering (Honours) in Software and

Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Signed: Aidan Malone Date: 31/03/2026

Acknowledgements

I would like to thank my supervisor Paul Lennon for the advice and mentorship throughout the academic year. With his guidance I was able to stay on track and adhere to my project timeline from start to finish.

I would also like to thank Pat Hurney and Niall O’Keeffe for their knowledge and insight into aspects of my project, giving me valuable technical information and direction at key stages of the development process.

Table of Contents

Declaration.....	2
Acknowledgements.....	3
Table of Contents.....	4
1 Summary	7
2 Poster	8
.....	8
3 Introduction	9
4 Background	11
4.1 The Drone Threat Landscape	11
4.2 YOLO Architecture and YOLO11n.....	12
4.3 Convolutional Neural Networks.....	13
4.4 ONNX and Runtime Optimisation	13
4.5 Kalman Filtering	14
4.6 Object Detection Evaluation Metrics	15
4.7 Servo Motor Control	15
5 Project Architecture	17
6 Project Plan	19
6.1 Agile Development Methodology	19
6.2 Semester 1 Summary	19
6.3 Semester 2 Development Sprints	20
6.4 Technical Challenges and Resolutions	21
7 Dataset Preparation and Model Training.....	22
7.1 Dataset Selection and Justification	22
7.2 Training Environment.....	23
7.3 Training Configuration	23
7.4 Training Results and Evaluation	24
7.5 Model Export and Deployment.....	26
8 Detection Pipeline	27
8.1 Camera Capture and Frame Preprocessing	27

8.2	YOLO11n ONNX Inference	27
8.3	Context-Aware Filtering Pipeline	28
8.3.1	Minimum Bounding Box Area Filter	28
8.3.2	Maximum Bounding Box Area Filter	28
8.3.3	Temporal Persistence Filter	29
8.4	Adaptive Confidence Thresholding	29
9	Object Tracking — Kalman Filter.....	31
9.1	Design Rationale.....	31
9.2	State Vector and Motion Model	31
9.3	Dynamic Time Step.....	32
9.4	Predict-Only Mode for Missed Detections.....	32
9.5	Integration with Servo Control	33
10	Detection Logging and Performance Metrics	34
10.1	CSV Logging System	34
10.2	Baseline Metrics Script.....	35
11	Response System.....	36
11.1	GPIO Buzzer Alarm	36
11.2	Pan-Tilt Servo Hardware Build	36
11.3	Servo Targeting Logic	38
11.4	Flask Remote Monitoring Stream	39
12	Testing and Results	40
12.1	Test Methodology	40
12.2	Batch 1 — Indoor Baseline.....	41
12.3	FPS Performance Across Configurations.....	41
12.4	Batch 2 — Outdoor Post-Filtering.....	42
12.5	Batch 3 — Full System Post-Servo Integration	44
13	Ethics	46
14	Conclusion	48
15	Appendix	50
15.1	GitHub Repository.....	50
15.2	Hardware Bill of Materials	50
15.3	Software and Library Versions	50
15.4	Sample CSV Log Extract.....	51

16	References.....	52
----	-----------------	----

1 Summary

This project aims to develop an efficient, low-cost drone detection system capable of operating in real-time on minimal edge hardware while maintaining accuracy and stability. The motivation behind this work is to address the need for rapidly deployable, temporary surveillance systems that can operate in newly secured or dynamic environments where infrastructure is limited, particularly in the context of real-time situational awareness and security monitoring.

The scope of this project includes a real-time drone detection system implemented natively on Raspberry Pi 4 hardware using a trained YOLO-based object detection model. It focuses on processing live camera input, performing inference for drone detection, and enhancing output stability using temporal filtering.

The system incorporates several important features, including real-time drone detection, confidence-based filtering, and a GPIO-based alarm system. Notably, it also includes a servo-based tracking response that allows a pan-tilt mechanism to physically orientate and target a detected drone.

To achieve project goals, a computer vision pipeline was implemented on edge hardware. This involved stages of frame capture, YOLO inference, detection filtering, object tracking and a triggered response. Additional software improvements such as temporal persistence, adaptive confidence thresholding, Kalman filtering, and rule-based decision logic were used to improve robustness, reduce false positives, and reduce missed detections.

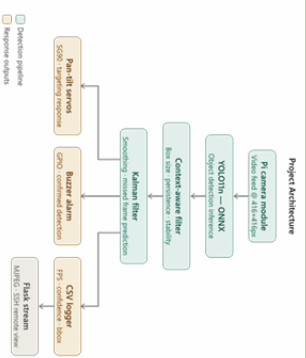
In conclusion, this project demonstrates that reliable real-time drone detection can be achieved on low-power edge systems through the combination of lightweight deep learning models and post-processing techniques. The system provides a foundation for further development in scalable, rapidly deployable surveillance and situational awareness applications.

Final Year Project

Atlantic Technological University

Bachelor of Engineering (Honours) in Software and Electronic Engineering

The motivation behind this work is to address the need for rapidly deployable, temporary surveillance systems operating in dynamic environments where infrastructure is limited. Existing counter-drone solutions demand significant cost and fixed infrastructure, making them impractical for resource-constrained scenarios. This project delivers a self-contained, low-cost drone detection and response system built on a Raspberry Pi 4 – no network required, operational within minutes.



Rapid deployment – self-contained, operational within minutes Real-time drone detection using YOLO10n (4.6+4.15px inference) Deployed on Raspberry Pi 4 – fully embedded, no cloud inference Kalman Filter for temporal smoothing and missed-frame prediction Context-aware filtering; temporal, persistence, bounding box size thresholds, box stability, and adaptive confidence Real-time servo system tracks detected drone using bounding box centre coordinates PiGPIO buzzer alarm triggers on confirmed detection	Figure 1. Drone detection and tracking system architecture
--	---



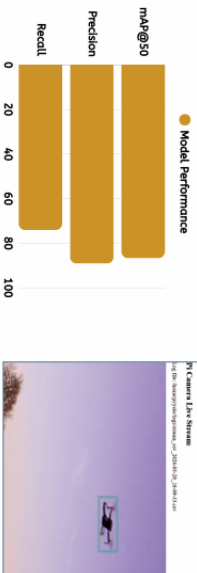
- 1 **Model Development** - Two datasets were evaluated: a Roboflow drone dataset (baseline) and a Datasethinja UAV dataset (final deployment model). Both were trained using YOLO11n in Google Colab. The final model was trained for 80 epochs at $416 \times 416 \times 3$ resolution using the Adamw optimiser

2 Embedded Deployment - Model exported to ONNX format for optimise CPU inference on Raspberry Pi 4, operating headlessly via SSH with a Flask MJPEG live stream.

3 Context-Aware Filtering - Raw detections pass through bounding box size thresholds, 3-frame temporal persistence, box stability checking, and adaptive confidence thresholding before a detection is confirmed.

4 Kalman Filter Tracking - A linear Kalman filter smooths bounding box positions and maintains tracking continuity across missed detection frames - critical at ~4 FPS.

5 Pan-Tilt Servo Response – On confirmed detection, the Kalman-predicted centre point is mapped to pan and tilt servo positions, with smoothing to reduce jitter.



The ONNM model, at 416px achieved a 4 FPS improvement over the baseline Pytorch model, reaching approximately 4 FPS on the Raspberry Pi 4. Context-aware filtering reduced false positives from 58.2 per minute to near-zero in outdoor conditions. The final trained model achieved a mAP@50 of 0.866 with a precision of 0.83, demonstrating reliable detection with low false alarm rates. Outdoor batch testing confirmed consistent drone detection at 5–10 metres with 60–80% confidence.

Software	Hardware
Python	Raspberry Pi 4
YOLOv11n (Ultralytics)	Raspberry Pi Camera Module v2
Google Colab	USB Power Supply
ONNX runtime	Sandisk Ultra 32GB Memory Card
Raspberry Pi OS	2 x 480 Servo Motors
OpenCL	GPIO buzzer
Git (Project Management)	Breadboard + Jumper Wires

This project successfully delivered a self-contained, rapidly deployable drone detection and response system on low-cost embedded hardware. Operating at approximately 4 FPS with near-zero false positives outdoors, the system demonstrates a viable alternative to costly fixed infrastructure solutions. Future work could explore higher-resolution inference on more capable hardware, multi-object tracking, and acoustic sensor fusion to further extend detection range and reliability.



3 Introduction

The motivation behind this work is to address the need for rapidly deployable, temporary surveillance systems that can operate in newly secured or dynamic environments where infrastructure is limited, particularly in the context of real-time situational awareness and security monitoring. Existing counter-drone solutions demand significant cost and fixed infrastructure, making them impractical for resource-constrained or fast-moving scenarios such as newly secured ground, forward operating bases, or temporary critical infrastructure protection. This project delivers a self-contained, low-cost drone detection and response system built on a Raspberry Pi 4 that requires no external network or cloud dependency and is operational within minutes of deployment.

Unmanned Aerial Vehicles (UAVs) have become a growing security concern across a range of environments. In 2018 a single drone incident at Gatwick Airport in shut down operations for 36 hours, affecting 140,000 passengers and costing an estimated £64 million in disruptions [8]. In conflict and contested ground scenarios, drones are increasingly used for surveillance, espionage, and even weaponisation. The challenge of detection is compounded by so-called dark drones, which operate without detectable RF emissions, making optical and acoustic detection a critical fallback capability [9]. The demand for robust, rapidly deployable drone detection systems has grown substantially as a result of these evolving threats.

Current commercial counter-drone systems address the threat through combinations of radar, RF analysis, acoustic sensing, and computer vision. While effective, these systems are expensive, require fixed infrastructure and specialist installation, and are not suitable for dynamic or resource-constrained deployment scenarios. The gap this project addresses is therefore not one of raw capability but of accessibility and deployability: delivering a detection and response capability that can be set up in minutes using a low-cost commodity platform.

The project addresses this through the development of a vision-based detection and response system running entirely on embedded hardware. The system uses a YOLO11n object detection model exported in ONNX format for optimised CPU inference, combined with a multi-stage context-aware filtering pipeline, a linear Kalman filter for tracking, and a pan-tilt servo system for physical targeting response. A GPIO buzzer provides audible alerting on confirmed detections, and all detection events are logged to CSV files for evaluation.

This report is structured as follows. Section 4 provides background on the underlying technologies and concepts. Section 5 describes the overall project architecture. Section 6 outlines the project plan and development timeline. Sections 7 through 11 detail the system implementation, covering dataset preparation, model training, the detection pipeline, Kalman filter tracking, and the response system. Section 12 presents testing methodology and results. Section 13 addresses ethical considerations and Section 14 concludes the report.

4 Background

4.1 The Drone Threat Landscape

Consumer and commercial drone usage has grown exponentially over the last decade. In the United States alone, over one million small drones are registered for civilian use, a number that has grown significantly in the past five years. Incident rates near restricted airspace have increased accordingly, with over 3,000 drone sightings or near-incursions recorded near US airports since 2021. In 2024 - 2025, the FAA recorded over 785,000 drone airspace violations [8].

Drone threats broadly fall into several categories. Airspace intrusions near airports and critical infrastructure risk mid-air collisions and flight disruptions. Espionage and surveillance operations using drone-mounted cameras or sensors have been documented near military installations and government facilities. Contraband smuggling via drone across borders and into prisons is an established and growing criminal technique. In conflict zones, drones have been weaponised as improvised ordinance delivery platforms.

A significant challenge for detection systems is the emergence of dark drones: UAVs that operate without RF transmissions, either by design or through deliberate modification of consumer hardware. Such drones cannot be detected using radio frequency scanning alone, making optical detection a critical component of any robust counter-drone system. The ease with which a commercial drone can be modified to operate without RF signals compounds this challenge further [9].

Current commercial solutions are expensive and require fixed infrastructure. Advanced radar, RF scanners, and acoustic sensor arrays can cost hundreds of thousands of euros to install and integrate. Many smaller sites, regional airports, and temporary installations do not have access to these systems, relying instead on ground staff to visually report sightings. This reactive approach means a drone is often only identified after disruption has already occurred. The project described in this report is motivated directly by this gap.

4.2 YOLO Architecture and YOLO11n

You Only Look Once (YOLO) is a family of real-time object detection neural networks first introduced by Redmon et al. in 2016. The core innovation of YOLO is treating object detection as a single regression problem rather than a two-stage process. The input image is divided into a grid, and for each grid cell the network simultaneously predicts bounding box coordinates, objectness scores, and class probabilities. This single-pass design allows YOLO to achieve detection speeds that are orders of magnitude faster than prior region proposal methods such as R-CNN [1].

The YOLO architecture has evolved considerably since its introduction. Modern variants such as YOLO11 use a backbone network for feature extraction, a Feature Pyramid Network (FPN) neck for multi-scale feature aggregation, and a decoupled detection head that separates classification and regression tasks. The decoupled head design, introduced in later versions, improves both training stability and final accuracy compared to coupled head designs.

YOLO11n is the nano variant of the YOLO11 family, designed specifically for deployment on resource-constrained hardware. It uses C3k2 convolutional skip connections, which improve feature reuse while minimising the parameter count. The model is approximately 2.6 million parameters in total, making it one of the smallest practical object detection models while maintaining competitive accuracy on standard benchmarks [1].

Loss calculation in YOLO11 uses three components. Bounding box regression loss uses Distribution Focal Loss (DFL), which replaces simple IoU-based regression with a probability distribution over discrete positional bins, producing more precise bounding box localisation particularly for small objects. Classification loss uses binary cross-entropy. Objectness loss uses a combination of confidence and IoU metrics. The DFL loss function is of particular relevance to drone detection because drones are typically small objects occupying few pixels in the image, where precise box localisation is more challenging.

4.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) underpin all modern visual object detection systems including YOLO. A CNN processes image data through a series of learnable filter kernels applied across the spatial dimensions of the input. Each filter kernel produces a feature map highlighting the presence of a particular spatial pattern such as an edge, corner, or texture. Successive convolutional layers combine these basic features into increasingly complex representations, ultimately building up the ability to recognise high-level object categories [5].

The convolution operation itself involves sliding a kernel matrix over the input image and computing the dot product between the kernel weights and the local image patch at each position. For a 3×3 kernel applied to an image of width W and height H , the output feature map has dimensions $(W - 2) \times (H - 2)$ for valid padding. Padding the input with zeros (zero-padding) preserves the spatial dimensions through the operation. The set of values in the kernel are the learnable parameters updated by backpropagation during training.

Pooling layers reduce the spatial resolution of feature maps between convolutional layers, decreasing computational cost and providing a degree of spatial invariance. Max pooling selects the maximum value in each local region, while average pooling computes the mean. The final layers of a CNN detection head are fully connected layers that map the extracted feature representations to the output predictions, which in YOLO's case are bounding box coordinates and class probabilities.

4.4 ONNX and Runtime Optimisation

The Open Neural Network Exchange (ONNX) format is an open standard for representing machine learning models, enabling trained models to be deployed using inference runtimes other than the framework in which they were trained. Exporting a trained PyTorch model to ONNX allows it to be run using the ONNX Runtime inference engine [6].

ONNX Runtime applies a set of graph optimisations at model load time. These include operator fusion, which merges sequences of operations into single optimised kernels; constant folding, which pre-computes subgraphs whose inputs do not change at runtime; and memory planning,

which minimises peak memory allocation during inference. On CPU hardware such as the Raspberry Pi 4, these optimisations consistently deliver faster inference times than the native PyTorch runtime.

A key constraint in ONNX export is that the model input shape is fixed at export time. This means the ONNX model compiled for 416×416 pixel input cannot directly accept 640×640 input, and vice versa. If the input resolution needs to change, the model must be re-exported from the original PyTorch weights at the desired resolution. This was a practical issue encountered during development and is documented in Section 6.

4.5 Kalman Filtering

The Kalman filter is a recursive Bayesian state estimation algorithm developed by Rudolf E. Kálmán in 1960. It provides optimal linear estimates of the state of a dynamic system in the presence of Gaussian noise, combining predictions from a motion model with incoming measurements to produce a posterior estimate that is more accurate than either alone [5].

The filter operates in two alternating phases. In the prediction step, the current state estimate is propagated forward in time using a state transition matrix F , producing a predicted state estimate and a predicted error covariance. In the update step, when a new measurement is available, a Kalman gain is computed that weights the relative uncertainty of the prediction and the measurement. The state estimate is then corrected by a term proportional to the difference between the actual measurement and the predicted measurement. This difference is known as the innovation or residual.

For bounding box tracking in object detection, the state vector is defined as: $x = [cx, cy, vx, vy, w, h]$, where cx and cy are the bounding box centre coordinates, vx and vy are the horizontal and vertical velocity estimates, and w and h are the bounding box width and height respectively. The measurement vector is $z = [cx, cy, w, h]$, directly corresponding to the bounding box outputs from the YOLO model. At low frame rates, the prediction step maintains tracking continuity through frames where the detector produces no output.

Several variants of the Kalman filter exist for non-linear systems. The Extended Kalman Filter (EKF) linearises the non-linear dynamics around the current estimate using a Jacobian, while the Unscented Kalman Filter (UKF) uses a set of sigma points to propagate the full distribution through the non-linearity. The Particle Filter represents the distribution as a set of weighted samples and can handle arbitrary noise distributions. For tracking a single drone in 2D image space at low frame rates, the linear Kalman filter is the most appropriate choice: drone motion in image space is well approximated as locally linear, and the lightweight computational requirement is essential on embedded hardware [5].

4.6 Object Detection Evaluation Metrics

Object detection models are evaluated using several standard metrics. Precision measures the fraction of detections that are correct, defined as the ratio of true positive detections to the total number of detections including false positives. Recall measures the fraction of ground-truth objects that are detected, defined as the ratio of true positives to the total number of ground-truth instances. A detection is considered a true positive if its predicted bounding box overlaps the ground-truth box by at least a threshold Intersection over Union (IoU) value.

Mean Average Precision (mAP) is the primary evaluation metric for object detection. mAP@50 computes the average precision at an IoU threshold of 0.5. mAP@50-95 averages the precision over ten IoU thresholds from 0.5 to 0.95 in steps of 0.05, providing a more stringent measure of bounding box localisation quality. For this project, mAP@50 is the primary metric of interest since accurate localisation is less critical than reliable detection in a security monitoring context.

4.7 Servo Motor Control

Servo motors are actuators that rotate to a commanded angular position based on a Pulse Width Modulation (PWM) signal. A standard servo accepts a 50Hz PWM signal where the pulse width encodes the target position. A pulse width of approximately 1ms commands the servo to its minimum position (0 degrees), 1.5ms commands the neutral position (90 degrees), and 2ms commands the maximum position (180 degrees). The SG90 is a compact, lightweight micro-servo widely used in embedded systems due to its low cost, low weight, and compatibility with 5V GPIO logic levels.

The gpiozero Python library provides a high-level interface to Raspberry Pi GPIO pins including a Servo class that abstracts the PWM signal generation. The library maps servo position to a normalised value in the range -1.0 to +1.0, where -1.0 corresponds to the minimum position, 0.0 to the neutral position, and +1.0 to the maximum position. This normalised interface simplifies the mapping from bounding box coordinate errors to servo commands in the tracking implementation.

5 Project Architecture

The system architecture follows a sequential pipeline from camera input to physical response output. Each stage of the pipeline passes its output directly to the next, with parallel outputs at the response stage triggering servo movement, buzzer alarm, and CSV logging simultaneously. Figure 1 illustrates the full pipeline.

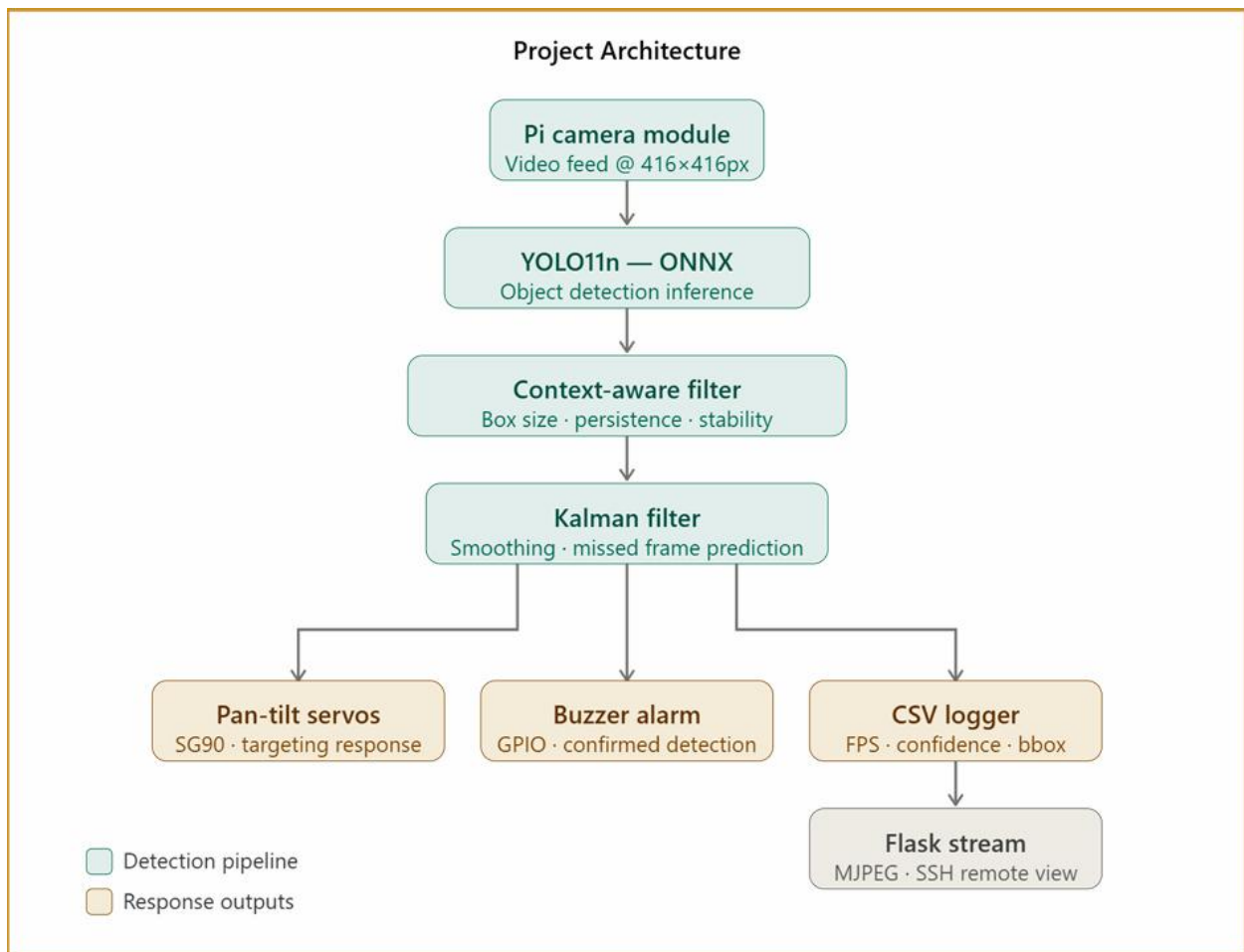


Figure 1: System Architecture Pipeline

The pipeline begins with the Raspberry Pi Camera Module V2 capturing frames at 416×416 pixels. Each frame is passed to the YOLO11n ONNX model for object detection inference. Raw detections output by YOLO are passed through a multi-stage context-aware filtering pipeline. Detections

that pass all filtering stages are accepted as confirmed detections. Confirmed detections are fed to the Kalman filter, which smooths positional outputs and maintains a continuous tracking estimate during frames where YOLO produces no detection.

The Kalman filter output drives three response components: the pan-tilt servo system for physical targeting, a GPIO buzzer for audible alerting, and a CSV logger for persistent data recording. A Flask-based MJPEG video stream transmits the annotated live feed over the local network for remote monitoring via SSH.

The codebase is structured as a set of modular Python files, each encapsulating a distinct pipeline stage. The camera module handles frame capture and colour space conversion. The detection module loads the ONNX model and performs inference. The filter module implements the context-aware filtering pipeline. The Kalman module implements the tracking filter. The servo module handles pan-tilt control. The logger module manages CSV file creation and per-frame data writing. The stream module implements the Flask MJPEG server. A main script imports and coordinates all modules in the detection loop.

Module	Responsibility
baseline_metrics.py	Post-session CSV log analysis and performance metrics calculation
camera.py	Frame capture from Pi Camera Module V2 using Picamera2
data_logging.py	Per-frame CSV data logging
detection.py	YOLO11n ONNX model loading and per-frame inference
main.py	Pipeline coordination and main detection loop
raw_frame_logger.py	Raw unfiltered frame data logging
servo_test.py	Servo motor testing and calibration utility
stream.py	Flask MJPEG live stream server

Table 1: Codebase Module Structure

6 Project Plan

6.1 Agile Development Methodology

The project was managed using an agile sprint-based methodology, with tasks, epics, and sprint goals tracked on a Jira board throughout both semesters. Each sprint typically spanned three to six weeks. Sprint reviews were performed against predefined completion criteria, and a new sprint was planned at the beginning of each week based on the state of outstanding tasks and any new requirements identified from testing or supervisor feedback.

Story points were used to estimate task effort, with one point corresponding approximately to one hour of work. An important early lesson in the project was that story points were not being used correctly in Semester 1: tasks were being marked as done without corresponding point allocation, leading to burndown reports that did not reflect actual effort. This was identified and corrected from Sprint 2 of Semester 2 onwards.

Version control was managed using Git, with a GitHub repository initialised at the start of Semester 2, Week 1, following the transition to headless SSH operation. The repository is publicly accessible at https://github.com/AidanM324/Drone_Detection_And_Response_System.git. Prior to this point, code was versioned informally through file backups, which was noted as a weakness in the semester 1 review.

6.2 Semester 1 Summary

Semester 1 ran from October to December 2025 and focused on environment setup, initial model training, and getting a functional live detection pipeline running on the Raspberry Pi 4. The majority of the time in this semester was spent resolving hardware and software configuration issues that are well documented in the project notebook.

Initial work included configuring the Raspberry Pi OS on the Pi 4, resolving a NumPy version conflict between TensorFlow and the YOLO runtime, and setting up a Python virtual environment for YOLO inference. The decision to use Ultralytics YOLO rather than TensorFlow Lite was made after comparing inference times: TFLite required a conversion pipeline and showed no clear

advantage for this use case, while Ultralytics provided a straightforward API and good documentation for Raspberry Pi deployment.

A baseline drone detection model was trained using a Roboflow drone dataset in Google Colab, producing both a PyTorch .pt model and an ONNX export. Both were successfully transferred to the Pi via USB and tested with a one-shot inference script. A critical finding was that the model produced a false detection on a bus image from the Ultralytics test set, indicating overfitting on the training data. This informed the decision to retrain with a larger and more diverse dataset in Semester 2.

By the end of Semester 1, a functional live detection script was running on the Pi with event-based logging. The camera was working correctly after several OS reflashes to resolve libcamera and picamera library corruption issues. The system was demonstrated at the semester 1 review, with feedback from the supervisor identifying FPS performance, false positive rate, and detection filtering as the key areas for Semester 2 improvement.

6.3 Semester 2 Development Sprints

Semester 2 consisted of seven sprints, each targeting specific technical improvements. The sprint structure and key deliverables are summarised in Table 2.

Sprint	Period	Key Deliverables
Sprint 1	Jan 2026	SSH headless operation, Flask stream, CSV structured logging, modularised codebase, GitHub initialised
Sprint 2	Jan-Feb 2026	Baseline metrics script, test scenarios defined, FPS and bounding box bug fixes, raw camera frame logging
Sprint 3	Feb 2026	ONNX model transition, min/max bounding box area filters, N-frame temporal persistence, adaptive confidence thresholding
Sprint 4	Feb-Mar 2026	DatasetNinja dataset extraction and conversion, model retrain at 416px, training results evaluation, Batch 1 indoor testing
Sprint 5	Mar 2026	Kalman filter implementation, Batch 2 outdoor testing, dataset domain gap analysis
Sprint 6	Mar-Apr 2026	Pan-tilt servo hardware build, servo targeting implementation, buzzer alarm, Kalman timing bug fix

Sprint 7	Apr 2026	Confidence delta logging, BMS enhancements, report and poster, Batch 3 testing
----------	----------	--

Table 2: Semester 2 Sprint Summary

6.4 Technical Challenges and Resolutions

Several significant technical challenges were encountered and resolved across the project lifecycle. Table 3 documents the most impactful issues and how they were addressed.

Challenge	Root Cause	Resolution
NumPy version conflict	TFLite required NumPy $\leq 1.19.5$; YOLO required newer	Downgraded NumPy, moved to Ultralytics YOLO
libcamera corruption	Incomplete Pi OS install, missing rpicas-apps	Multiple OS reflashes, manual rpicas-apps install
ONNX version mismatch	Colab exported newer opset than Pi ONNX Runtime supported	Re-exported model with lower opset version from Colab
Blue hue on live stream	Double BGR/RGB conversion in camera and detection modules	Removed duplicate conversion from camera module
Bounding box tensor format in CSV	YOLO returns PyTorch tensors; CSV writer expected plain Python types	Added .tolist() call on detection outputs before logging
Colab GPU limits during retrain	Free tier compute units exhausted	Purchased Colab compute units for T4 GPU access
Kalman dt bug	Previous timestamp not initialised before first frame	Initialised prev_time = time.time() at script startup
Servo PWM/buzzer interference	PWM clock sharing on Pi GPIO	Identified; mitigation planned (dedicated PWM controller)
Bounding box detection wrapping	Former Roboflow dataset didn't wrap detections properly	Identified issue with labelling in dataset, changed to a better dataset
Indoor testing	Indoor testing provided inconsistent results	Found that the majority of images in new dataset were outside, transitioned to outside testing

Table 3: Technical Challenges and Resolutions

7 Dataset Preparation and Model Training

7.1 Dataset Selection and Justification

Two datasets were used over the course of the project, reflecting the iterative model development process. The first dataset, sourced from Roboflow, was used to establish a working baseline model in Semester 1. The dataset contained labelled drone images and was straightforward to download and use. However, real-world testing revealed that the trained model was overfitting: it produced false detections on unrelated objects such as buses when tested with an Ultralytics sample image. The decision was made to retrain using a larger, more representative dataset.

The second dataset, sourced from DatasetNinja, was selected for the final deployed model. The dataset contains a larger number of annotated drone images across a wider variety of drone types, altitudes, angles, and environmental conditions. A review of randomly sampled images from the dataset confirmed that the majority of training images show drones against outdoor backgrounds including sky, tree canopy, and ground, with no indoor scenes. This is directly aligned with the intended deployment environment and explains the dramatic difference in false positive rate between indoor and outdoor testing.

The DatasetNinja dataset was supplied in a format using JSON annotation files with a custom structure. Extraction of the archive was complicated by file corruption during decompression, where the JSON annotation files were being written as empty files. This was diagnosed using a validated extraction pipeline that checked each file's integrity on write. Once correctly extracted, the annotations were converted from the JSON format to YOLO-format .txt files with one bounding box per line in the format [class_id cx cy w h], normalised to [0,1]. The directory structure was reorganised to the train/val split format expected by the Ultralytics training pipeline, and a data.yaml configuration file was created specifying the dataset paths and class names.

7.2 Training Environment

Training was conducted in Google Colab using a T4 GPU. The free tier of Colab imposed compute unit limits that were exhausted during the initial retraining attempts, leading to training sessions stalling without completing. This was resolved by purchasing Colab compute units, which provided uninterrupted access to the T4 GPU for the full training run.

The training script was written as a Colab notebook, importing the Ultralytics YOLO library and specifying the training configuration as parameters to the `model.train()` call. Training outputs including the `results.csv` metrics log, confusion matrix, precision-recall curve plots, and the `best.pt` and `last.pt` weight files were saved to Google Drive at the end of each session to ensure persistence across Colab session resets.

7.3 Training Configuration

The final training configuration for the deployed model was developed iteratively, beginning with a simple configuration and adding data augmentation techniques and training tuners based on analysis of the results from each run. Table 4 summarises the final configuration used.

Parameter	Value / Setting
Model architecture	YOLO11n (Nano variant, ~2.6M parameters)
Epochs	80
Input resolution	416 × 416 pixels
Batch size	16
Optimiser	AdamW
Learning rate schedule	Cosine decay (<code>cos_lr=True</code>)
Warmup epochs	3
Automatic Mixed Precision	Enabled (<code>amp=True</code>)
Mosaic augmentation	Enabled — combines 4 images into one training sample
MixUp augmentation	Enabled — blends pairs of images and labels
HSV augmentation	<code>hsv_h=0.015</code> , <code>hsv_s=0.7</code> , <code>hsv_v=0.4</code>
Multi-scale training	Enabled
Device	T4 GPU (Google Colab)

Table 4: Final Model Training Configuration

The 416px input resolution was selected as a compromise between inference speed and small object detection quality. At 640px, the exported ONNX model ran at approximately 2 FPS on the Pi 4 CPU, which is insufficient for real-time tracking. At 320px, drones beyond approximately 10 metres became too small in the frame for reliable detection. The 416px resolution achieves approximately 4 FPS while maintaining detection capability at distances relevant to the deployment scenario.

AdamW was selected over standard SGD because it converges faster on moderate-sized custom datasets. For the Roboflow baseline, standard SGD was used; switching to AdamW for the DatasetNinja retrain produced faster convergence and a smoother loss curve. The cosine learning rate schedule provides a smooth decay that avoids instability in late training epochs, which is beneficial when training for 80 epochs with a relatively small dataset.

Mosaic augmentation was a critical inclusion for small object detection performance. By combining four training images into a single sample, the model sees drones at multiple scales and in diverse spatial contexts within every batch. This has been shown to significantly improve detection of small objects compared to standard single-image augmentation. MixUp augmentation blends images and their labels, reducing the model's tendency to make overconfident predictions, which is beneficial for reducing false positives in deployment. HSV augmentation randomises hue, saturation, and brightness values, making the model robust to the variable lighting conditions encountered in outdoor real-world testing.

7.4 Training Results and Evaluation

The final training run over 80 epochs produced the performance metrics shown in Table 5. The training process was evaluated both through the raw metrics and through visual inspection of the training curve plots generated by the Ultralytics library.

Metric	Value
mAP@50	0.866
mAP@50-95	0.514
Precision	0.890
Recall	0.740

Table 5: Final Model Training Metrics

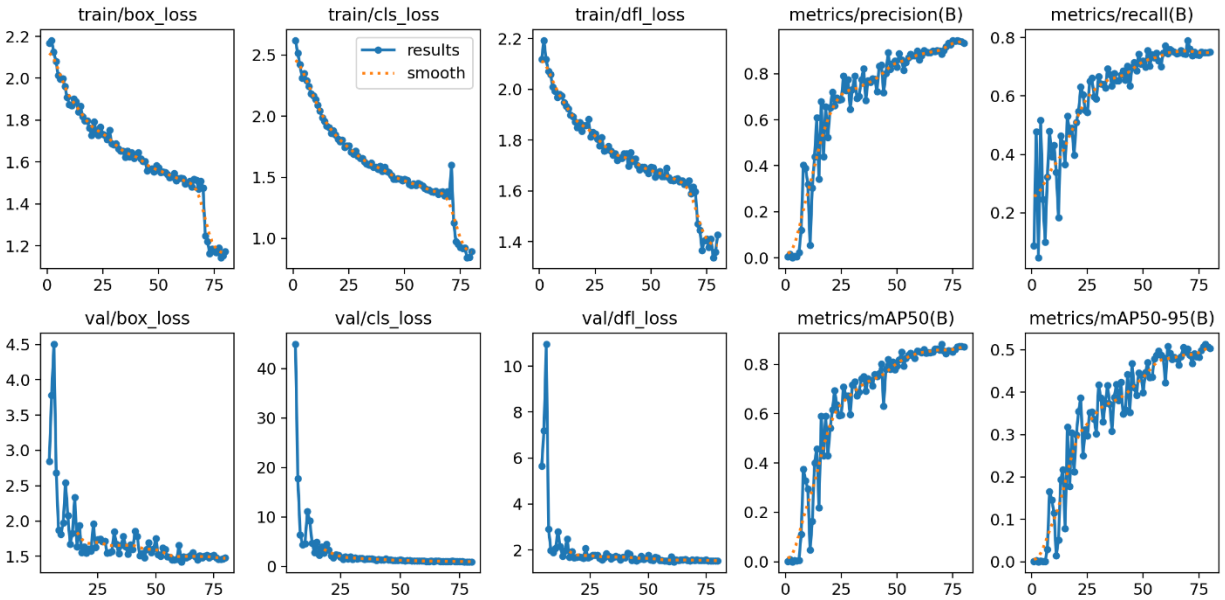


Figure 2: YOLO11n Training Performance Curves — mAP@50, Bounding Box Loss, Classification Loss, Precision, Recall

All three training loss components (bounding box loss, classification loss, and distribution focal loss) decreased smoothly across the full 80 epochs with no instability, confirming stable optimisation. Validation loss tracked training loss closely throughout training with no widening gap, indicating no significant overfitting. The training and validation curves plateaued around epoch 65–70, suggesting that 80 epochs was a slightly conservative but safe choice.

The precision of 0.89 indicates that when the model produces a detection with confidence above the threshold, it is correct approximately 89% of the time. In a security monitoring context this is an important property, as false positives cause unnecessary alarm responses and can erode operator trust. The recall of 0.74 means approximately 74% of drone appearances produce no

detection. In practice this manifests as brief dropout frames rather than sustained missed detection, since a continuously flying drone will re-enter the model's detection zone across successive frames. The Kalman filters predict-only mode and the temporal persistence counter both contribute to managing these short dropouts.

An initial custom visualisation of the validation losses produced anomalous straight-line increasing curves. This was identified as a data parsing bug in the visualisation code rather than a genuine training problem: the YOLO11 results.csv file stores validation metrics in a different column ordering from earlier versions, and the plotting code was indexing incorrect columns. The Ultralytics-generated built-in plots were used for all subsequent evaluation, as these are guaranteed to be parsed correctly.

7.5 Model Export and Deployment

Following training, the best.pt weights file was exported to ONNX format using the Ultralytics export function with imgs=416 and the target ONNX opset version set explicitly to ensure compatibility with the ONNX Runtime version available on the Raspberry Pi 4. The initial export used the default opset version, which was too new for the Pi's ONNX Runtime package, causing a runtime version mismatch error on first use. Re-exporting with a lower explicit opset version resolved this.

The exported ONNX file was transferred to the Raspberry Pi via USB. The file is loaded at startup by the detection module and held in memory for the duration of the detection session. ONNX Runtime was installed in the Python virtual environment on the Pi using pip, with the CPU-only variant to avoid unnecessary dependency overhead.

8 Detection Pipeline

8.1 Camera Capture and Frame Preprocessing

The Raspberry Pi Camera Module V2 captures frames using the libcamera stack on Raspberry Pi OS Bookworm. The Picamera2 Python library provides access to the camera from within the detection script. Frames are captured in BGR format by default through the OpenCV interface, and a colour space conversion from BGR to RGB is applied before passing the frame to YOLO, which expects RGB input.

This BGR/RGB conversion was the source of an early bug where the live stream appeared with a persistent orange-blue hue. Investigation identified that the conversion was being applied in both the camera capture module and the detection module simultaneously, resulting in a double conversion that effectively swapped the channels twice and corrupted the colour display. The fix was to centralise the conversion in the detection module only, removing it from the camera module.

Each frame captured from the camera is at native resolution and is resized to 416×416 pixels by the YOLO inference engine's internal preprocessing before model input. The frame is also written to the logging system and streamed to the Flask endpoint at this point in the loop.

8.2 YOLO11n ONNX Inference

Inference is performed by passing each frame to the loaded ONNX model through the Ultralytics YOLO Python API. The `model.predict()` call accepts the frame, the `imgsz` parameter set to 416, and the current adaptive confidence threshold. Verbose output is suppressed using `verbose=False` to avoid serial output overhead on the Pi.

The model returns a `Results` object for each frame. The bounding box coordinates, confidence scores, and class labels for all detections above the confidence threshold are extracted from this object. Bounding box coordinates are returned as PyTorch tensor objects. An early bug in the CSV logging system was caused by these tensor objects being written directly to the CSV file, producing entries such as `tensor(0.45)` rather than plain numeric values. This was resolved by

calling `.tolist()` on the bounding box output to convert it to a plain Python list before any logging or filtering operations.

At approximately 4 FPS, each inference call takes approximately 250ms. The dominant bottleneck is the ONNX model inference itself. The Flask stream encoding adds a small additional overhead, as does the CSV logging. Frame capture and preprocessing are fast relative to inference. Total round-trip latency from frame capture to confirmed detection output is approximately 300–400ms under normal operation.

8.3 Context-Aware Filtering Pipeline

Raw YOLO detections are passed through a multi-stage filtering pipeline before any detection is treated as confirmed. The filtering is applied in sequence, with a detection needing to pass every stage to be accepted. The stages are designed to address specific classes of false positive that were observed in baseline testing.

8.3.1 Minimum Bounding Box Area Filter

The first filter rejects detections whose bounding box area is below a minimum threshold. The threshold was set at 6,000 pixels squared, corresponding to approximately 2% of the 416×416 frame area. This threshold eliminates micro-detections caused by noise, background texture, or very distant objects that are too small to be reliably identified.

The threshold value was determined empirically during baseline testing. Examination of false positive detections in the no-drone indoor scenario revealed that many were extremely small bounding boxes overlapping with background texture elements. Setting the threshold at 6,000 pixels squared eliminated these without affecting detection of the drone at any realistic detection distance.

8.3.2 Maximum Bounding Box Area Filter

A complementary maximum area filter rejects detections above 130,000 pixels squared, corresponding to approximately 45% of the frame area. This filter targets anomalously large bounding boxes that are inconsistent with a drone at any plausible distance. These were

observed during initial testing, particularly when the model was producing large boxes in cluttered indoor environments.

The maximum threshold was set conservatively at 45% of the frame area rather than a tighter value, to avoid rejecting genuine close-range detections where a large drone might occupy a substantial portion of the frame. In practice, drone detections in test scenarios were all well below this threshold.

8.3.3 Temporal Persistence Filter

The temporal persistence filter requires that a detection passes all preceding filters in $N=3$ consecutive frames before it is accepted as a confirmed detection. A persistence counter is incremented each frame a valid detection is present and reset to zero on any frame with no valid detection. The confirmed detection flag is set to 1 only when the counter reaches N .

This single filter had the largest individual impact on false positive rate of all the filtering stages. Sporadic single-frame or two-frame detections caused by background objects, transient reflections, or motion artifacts are entirely eliminated by the requirement for three consecutive valid frames. The value of $N=3$ was chosen as a balance between eliminating sporadic false positives and avoiding excessive latency in confirming a genuine drone detection. At 4 FPS, $N=3$ corresponds to a confirmation latency of approximately 750ms from first detection.

8.4 Adaptive Confidence Thresholding

A fixed confidence threshold does not account for the temporal context of the detection scenario. When a drone is actively present and being tracked, the system benefits from a higher threshold that reduces the chance of confirming background noise as a secondary detection. When the system is in an idle state with no drone present, a lower threshold ensures early detections are not missed.

The adaptive thresholding system initialises with a base confidence level of 0.30 and maintains a current confidence value. When confirmed detections occur, the current confidence is incremented by a small amount per frame. When no detection is present, the current confidence decays toward the base value over time. The current threshold and the confidence delta (the

difference between the highest detection confidence in the frame and the threshold) are logged to the CSV file on every frame.

The rationale for a confidence delta field in the logging system is to provide retrospective visibility into near-threshold events. Frames where the raw YOLO detection confidence was slightly below the threshold but might have been a genuine detection are identifiable through this field, which is useful for calibrating the threshold parameters during evaluation.

9 Object Tracking — Kalman Filter

9.1 Design Rationale

At approximately 4 FPS, the detection pipeline has limited temporal resolution. Between successive frames, a drone in motion can shift its position by a significant number of pixels, and the bounding box output from YOLO inherently contains frame-to-frame jitter due to detection uncertainty. Without smoothing, this jitter causes visible flickering in the bounding box overlay and, more critically, erratic servo movement that prevents the pan-tilt system from maintaining a stable target lock.

Missed detections are also a significant problem at low frame rates. YOLO may fail to detect the drone in a given frame due to motion blur, momentary occlusion, or confidence falling below the adaptive threshold. Without a tracking mechanism, these missed frames cause the confirmed detection flag to reset, triggering a servo return to neutral and a full re-acquisition cycle on the next detection. This produces visible instability in the servo behaviour and creates gaps in the alarm response.

A Kalman filter was selected to address both issues simultaneously. By maintaining a continuous state estimate of the drone's position and velocity, the filter produces smooth bounding box positions for the overlay and servo control and maintains the estimate through missed detection frames using prediction alone.

9.2 State Vector and Motion Model

The Kalman filter uses an 8-dimensional state vector $x = [cx, cy, w, h, vx, vy, vw, vh]$, tracking the bounding box centre coordinates, dimensions, and their corresponding velocity estimates. This allows the filter to track both the drone's position and changes in its apparent size as it moves toward or away from the camera. The measurement vector $z = [cx, cy, w, h]$ corresponds directly to the bounding box output from YOLO. The transition matrix models constant velocity motion, predicting each value at the next time step as its current value plus velocity multiplied by dt . Process noise Q and measurement noise R are diagonal matrices scaled by $1e-2$ and $2e-1$ respectively, balancing smoothness against responsiveness.testing.

9.3 Dynamic Time Step

A critical implementation detail is the use of a dynamic dt computed from real wall-clock timestamps rather than an assumed fixed frame interval. At a consistent 4 FPS a fixed $dt = 0.25$ seconds would be reasonable, however the Raspberry Pi 4's inference time varies frame by frame depending on CPU load, background system processes, and Flask stream encoding overhead. Frame intervals in practice range from approximately 0.2 seconds to over 0.5 seconds.

Using a fixed dt under these conditions produces inaccurate velocity estimates. If the actual frame interval is 0.5 seconds but the filter assumes $dt = 0.25$, the predicted position after a gap will be only half the true displacement, causing the tracking estimate to lag behind the actual drone position. The implementation clamps dt to the range $[0.01, 0.5]$ seconds to guard against anomalous values at initialisation or after system pauses.

The initial implementation contained a bug where the previous timestamp variable was not initialised before the first frame was processed, causing a divide-by-zero error. This was fixed by initialising `self.last_time = time.time()` in the constructor before the main detection loop begins.

9.4 Predict-Only Mode for Missed Detections

When a frame contains no valid detection, the Kalman filter enters predict-only mode. The state transition is applied to advance the state estimate forward in time without a measurement update, and the predicted position continues to follow the last known velocity vector of the drone. A `missed_frames` counter is incremented on each consecutive frame with no detection.

If the `missed_frames` counter exceeds a defined `max_missed_frames` threshold, the filter performs a full reset: `kalman_initialized` is set to `False`, the persistence counter and confidence values are cleared, and the bounding box output is zeroed. This prevents the filter from drifting indefinitely on a stale velocity estimate when the drone has genuinely left the frame. On the next confirmed detection the filter is reinitialised from scratch using `_init_kalman_state()`, which seeds the state vector directly from the new YOLO measurement with zero initial velocity.

9.5 Integration with Servo Control

The Kalman filter output (smoothed centre coordinates c_x , c_y) is used as the input to the servo targeting system rather than raw YOLO bounding box coordinates. This is a deliberate design decision: raw YOLO coordinates have frame-to-frame jitter that would cause the servo motors to hunt continuously, producing oscillation and poor targeting accuracy. The Kalman-smoothed coordinates change gradually and predictably, allowing the servo to track smoothly.

During batch 2 testing, the qualitative difference in servo tracking behaviour between raw YOLO coordinates and Kalman-smoothed coordinates was clearly visible. With raw coordinates, the servo oscillated continuously. With Kalman coordinates, the servo tracked the drone's position with visible smoothing.

10 Detection Logging and Performance Metrics

10.1 CSV Logging System

A structured CSV logging system records per-frame data throughout the duration of each detection session. A new CSV file is created at script startup with a timestamp-derived filename. Each row in the log corresponds to one processed frame and is written at the end of each detection loop iteration. The system logs both raw YOLO outputs and post-filtering confirmed detection outputs, enabling retrospective analysis of filtering effectiveness.

The decision to log at the frame level rather than the event level was made in Semester 2 Week 2 after analysing the limitations of the event-based logging from Semester 1. Frame-level logging preserves the full temporal structure of the detection session, making it possible to compute metrics such as false positives per minute, missed detection rate, and FPS statistics over any defined time window. Event-only logging would miss the idle frames between detections, making it impossible to compute the false positive rate accurately.

Field	Description
timestamp	ISO 8601 datetime of frame capture
frame_id	Sequential integer frame counter from session start
detected	1 if detection passed all filtering stages; 0 otherwise
raw_detected	1 if YOLO produced any bounding box above threshold; 0 otherwise
confidence_threshold	Current adaptive confidence threshold value at this frame
confidence	Highest YOLO confidence score in frame (0.0 if no detection)
confidence_minus_threshold	Confidence minus threshold (positive = above threshold)
bbox_x1	Bounding box top-left x coordinate (pixels)
bbox_y1	Bounding box top-left y coordinate (pixels)
bbox_x2	Bounding box bottom-right x coordinate (pixels)
bbox_y2	Bounding box bottom-right y coordinate (pixels)
bbox_area	Bounding box area in pixels squared
fps	Instantaneous frames per second for this frame

Table 6: CSV Log Fields and Descriptions

10.2 Baseline Metrics Script

A separate Python analysis script (`bms.py`) was developed to process saved CSV log files and compute standardised performance metrics. The script uses the Pandas library for data loading and manipulation, which provides an efficient interface for filtering and aggregating the frame-level data.

The false positives per minute metric is computed by identifying time windows where no drone was physically present (from the test scenario timestamps), counting confirmed detection events within those windows, and dividing by the elapsed time in minutes. Missed detections are computed as the fraction of frames within a drone-present window where `confirmed_detected = 0`. Average FPS is computed as the mean of the `fps` column across the full session. Confidence threshold statistics including minimum, maximum, and mean values across the session are also extracted.

A later enhancement added computation of the `conf_delta` distribution to the metrics script. This provides visibility into how far the highest detection confidences in each frame were from the adaptive threshold, enabling analysis of whether the adaptive thresholding is set appropriately for the test conditions.

11 Response System

11.1 GPIO Buzzer Alarm

A passive buzzer was connected to the Raspberry Pi 4 using GPIO pin 18 (BCM numbering) as the signal pin and pin 6 as the ground reference. The gpiozero library's TonalBuzzer class generates a PWM-based tonal output. The buzzer is activated when the confirmed detection flag transitions from 0 to 1 and remains active for the duration of the detection. When the confirmed detection flag falls back to 0, the buzzer is deactivated.

A known issue was identified during integration with the servo system: the PWM signal used to drive the servo motors interferes with the buzzer output, producing ringing, oscillation, or frequency modulation of the alarm tone rather than a clean sound. This is attributable to contention for the Raspberry Pi's hardware PWM clock resource, which is shared across all PWM-enabled GPIO pins.

Two mitigation strategies were identified. The first is to use hardware PWM pins exclusively for the buzzer and drive the servos from a dedicated external PWM controller such as the Adafruit PCA9685, which communicates over I2C and generates its own PWM signals independently of the Pi's GPIO. The second is to use different PWM frequencies for the buzzer and servos to reduce harmonic interference. The first approach is the cleaner solution but requires additional hardware procurement. This is documented as a known limitation of the current prototype implementation.

11.2 Pan-Tilt Servo Hardware Build

A pan-tilt assembly was constructed using two SG90 micro servo motors. One servo (the pan servo) provides horizontal rotation, and the second (the tilt servo) is mounted to the rotating arm of the pan servo to provide vertical rotation. This arrangement allows the system to point toward any location in the upper hemisphere of the camera's field of view.

The physical assembly required several steps. The servo blade of the tilt servo was paired down with a file to remove a raised centre section that would have prevented a flat mating surface for

supergluing. Both mating surfaces were cleaned with nail polish remover to degrease them before the adhesive was applied. The tilt servo was then superglued to the rotating arm of the pan servo and allowed to cure fully before testing.

Power for the servos was sourced by stripping a USB cable and using the 5V and GND conductors. This design decision was made to avoid drawing servo current through the Raspberry Pi 4's GPIO 5V pins, which have a shared current limit with the processor supply and can cause system instability or resets under servo load. The USB power cable provides a dedicated power path from the USB power supply directly to the servo power bus on the breadboard, with a shared ground connection to the Pi.



Figure 3: Pan-Tilt Servo Assembly and Hardware Platform

The servo signal wires were connected to GPIO pins 17 and 27 for pan and tilt respectively. Before connecting the servos, the polarity of the stripped USB power wires was verified by briefly connecting each wire to an LED with a current-limiting resistor. Reverse polarity on a servo will destroy it. The LED test confirmed correct polarity before the servos were connected.

An initial failure to get the servos working after assembly was traced to a missing ground connection between the servo GND bus on the breadboard and the Pi's GPIO GND pin. Without this common ground, the PWM signal reference level is undefined and the servo driver IC receives an indeterminate signal. Connecting the common ground resolved the issue immediately.

11.3 Servo Targeting Logic

The servo targeting system converts the Kalman-smoothed bounding box centre coordinates to servo position commands. The conversion process involves the following steps, applied each frame in which a confirmed detection is present.

First, the Kalman filter's smoothed centre position (cx , cy) is normalised to the range $[0, 1]$ by dividing by the frame width and height respectively. This produces a normalised position $nx = cx / \text{frame_width}$ and $ny = cy / \text{frame_height}$.

```
# Normalise (0 → 1)
nx = cx / frame_w
ny = cy / frame_h
```

Second, the normalised position is converted to a servo target value in the range $[-1, +1]$, where 0 corresponds to the frame centre. Both axes use the formulation $(0.5 - n) * 2$, which inverts the direction relative to the standard mapping. This was required because the physical servo mounting caused the pan axis to track away from the target without the inversion.

```
# Convert to servo targets (-1 → 1)
target_pan = (0.5 - nx) * 2
target_tilt = (0.5 - ny) * 2
```

Third, exponential smoothing with a coefficient $\alpha = 0.2$ is applied to the servo position update. Rather than commanding the servo directly to the target position, the position is updated incrementally toward the target by a fraction α of the error. This prevents abrupt servo movements in response to single-frame noise in the Kalman estimate and produces visibly smooth tracking behaviour. The value of $\alpha = 0.2$ was selected empirically; smaller values

produce smoother but more sluggish tracking, while larger values produce faster but more jittery response.

When no confirmed detection is present, the servos are commanded to return to the neutral position (`pan.value = 0`, `tilt.value = 0`) at each frame. This returns the camera assembly to a forward-facing orientation between detections.

11.4 Flask Remote Monitoring Stream

A Flask web server running on the Raspberry Pi provides an MJPEG live stream accessible from any browser on the local network via HTTP. The stream URL is `http://[pi-ip-address]:5000/video`. The stream displays the annotated video feed with bounding boxes rendered by OpenCV.

This was implemented to replace the original VNC viewer remote desktop approach used in Semester 1. VNC viewer consumed significant CPU resources on the Pi for rendering the graphical desktop environment, leaving less processing headroom for YOLO inference. The SSH-only headless operation with a Flask stream reduced overhead substantially: the Pi only needs to encode and stream MJPEG frames, which is far less resource-intensive than rendering a full desktop.

The Flask stream was implemented based on the Random Nerd Tutorials MJPEG streaming guide [11], adapted to integrate with the Ultralytics detection annotation output. The stream generator function runs in a separate thread from the detection loop, maintaining a shared frame buffer that is updated by the detection thread and read by the stream thread on each MJPEG frame request.

12 Testing and Results

12.1 Test Methodology

Testing was structured as a series of batches, with each batch conducted after a set of system improvements had been implemented. This approach provides clear before-and-after comparisons that quantify the effect of each improvement on the key performance metrics. Four test scenarios were defined at the start of Semester 2 and used consistently across all batches to enable direct comparison.

Scenario	Duration	Purpose
No Drone Present	2–4 minutes	Measure false positive rate in the absence of any target
Drone Stationary (5–10m)	2–4 minutes	Measure detection consistency and confidence stability at close range
Drone Moving (10–30m)	1–2 minutes	Measure detection robustness under motion and varying distance
No Drone, Moving Objects	2 minutes	Measure false positive rate in a complex scene with background motion

Table 7: Test Scenarios

For each test session, the CSV log file produced by the detection script was processed by the baseline metrics script (bms.py) to extract average FPS, false positives per minute, and missed detection percentage. Test timestamps were manually recorded for each scenario to allow the metrics script to compute statistics over the correct time window.

Due to weather conditions, outdoor testing was not always possible on the target date. Batch 1 was conducted indoors as a baseline test. Batch 2 was conducted outdoors following completion of the filtering pipeline and model retrain. Batch 3 was also conducted outdoors following completion of the project.

12.2 Batch 1 — Indoor Baseline

Batch 1 was conducted indoors in a university computing lab environment before any context-aware filtering had been implemented. The purpose was to establish a quantitative baseline against which subsequent improvements could be measured. The test environment included computers, keyboards, displays, and people moving in the background, representing a challenging scene for a model trained exclusively on outdoor imagery.

Scenario	PyTorch .pt Model (~1 FPS)	ONNX Model (~2 FPS)
No Drone Present	FP rate: ~58.2 /min	FP rate: similar (~58 /min)
Drone Stationary	Confidence: 64–77%	Confidence: 73–77%
No Drone, Moving Objects	FP rate: 58.2 /min	FP rate: comparable

Table 8: Batch 1 Indoor Baseline Results

The high false positive rate in the no-drone indoor scenarios was consistent with the dataset domain mismatch: the model was trained entirely on outdoor images and had not been exposed to the object textures, shapes, and spatial relationships present in an indoor environment. Moving arms, computer screens, and background clutter all triggered detections. However, the stationary drone test showed that the model could reliably detect the drone itself when present.

The ONNX model at this stage showed approximately 2x FPS improvement over the PyTorch model (approximately 2 FPS vs 1 FPS) with comparable detection quality. This confirmed the value of the ONNX export for performance improvement on the Pi.

12.3 FPS Performance Across Configurations

FPS performance was measured across all three model configurations to document the effect of each optimisation. Table 9 and Figure 4 summarise the progression.

Configuration	Average FPS on Raspberry Pi 4
PyTorch .pt model, 640px input	~1 FPS

ONNX model, 640px input	~2 FPS
ONNX model, 416px input (deployed)	~4 FPS

Table 9: FPS Performance Across Model Configurations

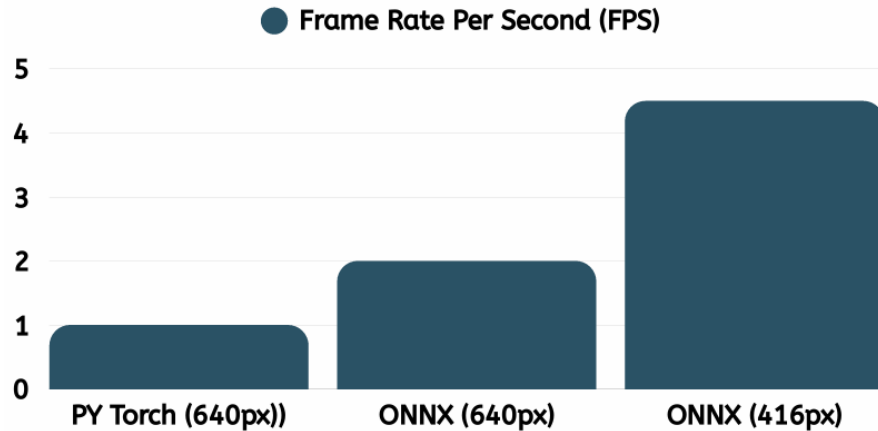


Figure 4: FPS Improvement Across Model and Resolution Configurations

The transition from PyTorch to ONNX at 640px provided a 2x improvement attributable entirely to ONNX Runtime graph optimisations. The further reduction to 416px input resolution provided an additional 2x improvement, giving a combined 4x improvement over the starting point. This trade-off in resolution was acceptable because drone detection performance at close range was maintained, and the reduced resolution only significantly affected detection capability at distances beyond 20–30 metres.

12.4 Batch 2 — Outdoor Post-Filtering

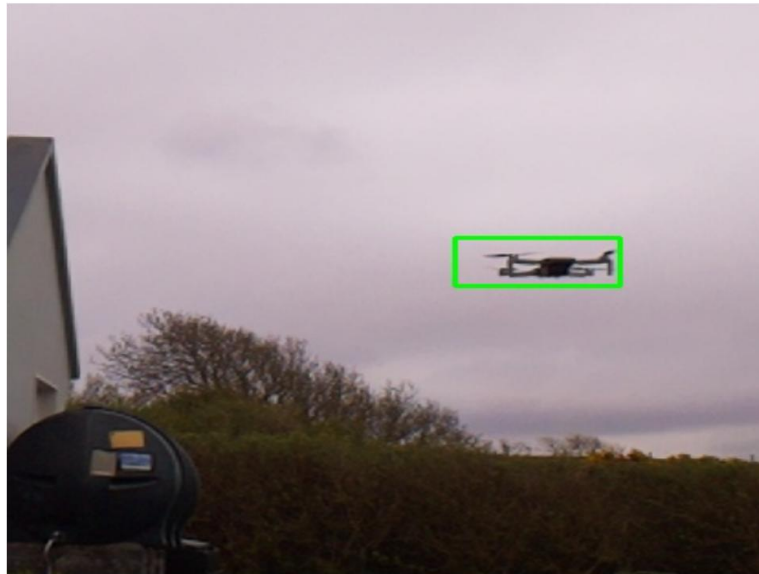
Batch 2 was conducted outdoors following implementation of the full context-aware filtering pipeline, the DatasetNinja model retrain, and the Kalman filter. The outdoor environment consisted of an open area with clear sky background. Testing was conducted in the afternoon with moderate overcast lighting; a subsequent test in lower light (evening) revealed the lighting sensitivity issue documented below.

Scenario	Outcome
No Drone, Outdoor (2 min)	0 false positives. Clean sky background produced zero YOLO detections.

Drone Stationary at 5–10m	Consistent frame-per-frame detection. Confidence 60–80%. No false positives.
Drone Moving at 10–30m	Intermittent detections. Missed detections increased beyond 20m. Lighting degradation a factor.
Dataset Domain Check	Random sample of training images confirmed predominantly outdoor, no indoor scenes.

*Table 10: Batch 2 Outdoor Testing Results***Pi Camera Live Stream**

Log file: /home/pi/yolo/logs/stream_csv_2026-04-25_13-43-07.csv

*Figure 5: Flask Live Stream — Drone Detection with Bounding Box*

The near-zero false positive rate in outdoor conditions with no drone present represented a dramatic improvement over the batch 1 indoor baseline. The clean sky background provided very little for the model to incorrectly classify, and the filtering pipeline eliminated any marginal detections. This confirmed that the system performs correctly in its intended deployment environment.

The consistent detection at 5–10 metres with 60–80% confidence validated the model and pipeline for close-range operation. Confidence values in this range are above the 0.60 threshold and comfortably above the temporal persistence counter requirement, meaning detections were confirmed reliably at this distance.

Beyond 20–30 metres, missed detections increased substantially. Two contributing factors were identified. First, at 416px resolution, a drone at 25 metres occupies approximately 20–30 pixels in each dimension, which is at the lower limit of reliable detection for YOLO models. Second, the test session was conducted as the sun was setting, and the deteriorating lighting conditions reduced image contrast, making the drone harder to distinguish from the sky background.

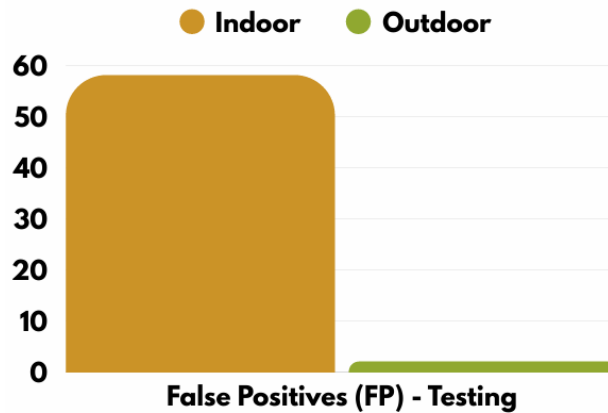


Figure 6: False Positive Rate Comparison — Before and After Context-Aware Filtering

12.5 Batch 3 — Full System Post-Servo Integration

Batch 3 testing was conducted outdoors on 25 April 2026, following integration of the servo targeting system and buzzer alarm. Four test logs were recorded across three scenarios, with the stationary drone scenario repeated after a confidence threshold adjustment following a poor initial detection rate.

The no-drone test recorded zero confirmed and zero raw detections across 407 frames over 112 seconds, maintaining the zero false positive rate established in batch 2. The first stationary drone test produced only 7 confirmed detections from 580 frames despite 208 raw detections being present, a missed detection rate of 98.8%, indicating the confidence threshold was set with no max limit. After lowering the threshold the same scenario produced 309 confirmed detections from 498 frames, reducing the missed detection rate to 38.0% with a mean confidence of 0.721. The moving drone test recorded 146 confirmed detections from 384 frames at a missed detection rate of 62.0%, with confidence ranging from 0.34 to 0.87 reflecting the increased difficulty of detecting a drone in motion.

Compared to batch 2, detection performance improved across both scenarios once the threshold was correctly calibrated. The stationary missed detection rate improved from 64.7% to 38.0% and the moving missed detection rate from 82.1% to 62.0%, while the zero false positive rate was maintained throughout. The key finding is that confidence threshold calibration is a significant operational parameter: the system is capable of strong detection performance in outdoor conditions, but the base threshold must be tuned to the specific lighting and distance conditions of each deployment.

Metric	Batch 2 Result	Batch 3 Result
Average FPS	3.46–3.92 FPS	3.57–3.78 FPS
FP/min (No Drone, Outdoor)	~0	~0
Confirmed detections/min (Stationary)	69.78	130.43
Missed detection rate (Stationary)	64.7%	38.0%
Missed detection rate (Moving)	82.1%	62.0%
Servo Response Accuracy		
Mean confidence (Stationary)	0.637	0.721
Mean confidence (Moving)	0.836	0.688

Table 11: Batch 3 Results Comparison

13 Ethics

A number of ethical considerations apply to this project and to the broader field of automated drone detection and surveillance systems.

The most significant concern relates to privacy. A camera-based detection system capable of continuous real-time monitoring could, if deployed without appropriate controls, be used for surveillance of individuals rather than strictly drone detection. In the context of this project, no video data is stored persistently: only detection metadata is written to CSV log files, and the live stream is accessible only on the local network behind a password-controlled SSH session. However, any future deployment of such a system in a public or semi-public area would require explicit legal authorisation under applicable national legislation, a privacy impact assessment, and compliance with GDPR data protection requirements.

The response mechanism introduces an additional consideration. The servo-based targeting system physically orientates the assembly toward a detected drone. In the current prototype, this is purely a passive tracking and alerting capability. Any future escalation to active countermeasures such as RF jamming, GPS spoofing, or directed energy would carry serious legal implications under aviation law, electronic communications regulations, and potentially national security legislation. Operating a jamming device without authorisation is illegal in virtually all jurisdictions regardless of the target.

There is also a question of accountability in automated detection systems. If the system produces a false positive in a security context, a response may be triggered against a legitimate drone operator acting within their rights. The high false positive rate observed in indoor testing demonstrates that model errors are a real concern. Any deployment of the system in a live security environment should include human review of alerts rather than fully automated response actuation.

The environmental cost of training deep learning models should also be acknowledged. Training sessions on GPU hardware such as Google Colab's T4 consume significant electrical energy. In

contrast, inference on the Raspberry Pi 4 consumes very little power relative to server-class hardware, which is a positive characteristic of the edge deployment model adopted by this project.

The datasets used in this project contain only drone and UAV imagery sourced from public repositories. No images of identifiable individuals were used in model training. The DatasetNinja and Roboflow datasets are publicly available for research and educational use.

14 Conclusion

This project successfully developed a fully embedded, real-time drone detection and response system operating on a Raspberry Pi 4 with no cloud dependency or external network requirement. Starting from a basic YOLO inference prototype in Semester 1, the system was evolved through iterative sprint-based development over two semesters into a production-quality pipeline incorporating a trained YOLO11n model, a multi-stage context-aware detection filter, a linear Kalman tracker, a pan-tilt servo targeting mechanism, a GPIO buzzer alarm, a Flask-based remote monitoring stream, and a comprehensive CSV telemetry logging system.

The key technical outcomes of the project are as follows. The transition from a PyTorch .pt model at 640px input to an ONNX-optimised model at 416px achieved a 4x improvement in inference throughput from approximately 1 FPS to approximately 4 FPS on the Pi 4 CPU. The context-aware filtering pipeline reduced the false positive rate from 58.2 false alarms per minute in unfiltered indoor testing to near-zero in outdoor realistic conditions. The DatasetNinja-trained YOLO11n model achieved a mAP@50 of 0.866 with a precision of 0.89 and recall of 0.74 on the validation set. The Kalman filter significantly improved bounding box stability and maintained tracking continuity through the short detection dropout frames that are inevitable at the system's low frame rate.

The primary limitation of the current system is detection consistency under varying environmental conditions. Batch 3 testing demonstrated that confidence threshold calibration is a significant operational parameter — with an incorrectly set threshold, missed detection rates of up to 98.8% were observed despite the drone being visible to the model. Once correctly calibrated, the system achieved a 38% missed detection rate on a stationary drone and 62% on a moving drone, while maintaining zero false positives throughout. Detection performance also degrades with distance; at 416px resolution, reliable detection becomes inconsistent beyond 20–30 metres. This is a direct consequence of the resolution trade-off required to achieve real-time performance on the Pi 4 CPU, and the system's performance envelope is appropriate for close-range perimeter protection in temporary secured zones.

Future development opportunities are substantial. Detection range limitations could be addressed by evaluating higher-resolution inference on more capable embedded hardware such as the Raspberry Pi 5 or a dedicated NPU accelerator board. Multi-object tracking using algorithms such as SORT would extend the system to handle simultaneous detection of multiple UAVs. Acoustic sensor integration using a microphone and DSP pipeline for drone motor frequency detection, originally planned as part of the project scope, represents a natural extension that could improve long-range detection through audio-visual sensor fusion. The servo targeting system could be improved through a dedicated PCA9685 PWM controller to eliminate buzzer interference, and through PID control to replace the current exponential smoothing approach for improved transient response.

The project demonstrates that reliable, low-latency drone detection and response is achievable on low-cost commodity embedded hardware through careful selection and combination of lightweight machine learning models, post-processing filtering, and classical state estimation techniques. The system as delivered provides a viable and practically deployable first line of detection for scenarios where cost, deployability, and self-sufficiency are the primary constraints.

15 Appendix

15.1 GitHub Repository

The full project codebase is available at:

https://github.com/AidanM324/Drone_Detection_And_Response_System.git

The repository contains the detection pipeline source files (camera.py, detection.py, filter.py, kalman.py, servo.py, logger.py, stream.py, main.py), the baseline metrics analysis script (bms.py), the Google Colab training notebook, sample CSV log files from batch 1, batch 2 and batch 3 testing, and the data.yaml configuration file for dataset training.

15.2 Hardware Bill of Materials

Component	Specification	Purpose
Raspberry Pi 4 Model B	4GB RAM, ARM Cortex-A72 @ 1.8GHz	Main processing unit
Raspberry Pi Camera Module V2	8MP Sony IMX219, fixed focus	Live video capture
SG90 Micro Servo Motor (x2)	180° range, 4.8–6V, ~9g	Pan-tilt targeting assembly
Passive Buzzer	GPIO-compatible, 3.3V/5V	Audible detection alarm
Breadboard + Jumper Wires	Full-size breadboard	Circuit integration and prototyping
SanDisk Ultra 32GB microSD	Class 10, 100MB/s read	Pi OS and data storage
USB Power Supply	5V 3A output	Raspberry Pi power
USB cable (stripped)	5V / GND conductors	Dedicated servo motor power

Table 12: Hardware Bill of Materials

15.3 Software and Library Versions

Tool / Library	Version / Notes
Python	3.11 (Raspberry Pi OS Bookworm)
Ultralytics YOLO	Latest (training and export)

ONNX Runtime	CPU-only build for Pi
OpenCV (cv2)	Pi-compatible build via pip
NumPy	Compatible version for YOLO/ONNX
gpiozero	v2.0.1
Flask	Latest stable
Picamera2	Pi Camera Module interface
Pandas	Data analysis for BMS script
Google Colab	T4 GPU environment for training
Jira	Sprint and backlog management
GitHub / Git	Version control from Semester 2 W1
VS Code SSH Remote	Remote development on Pi headless

Table 13: Software and Tool Versions

15.4 Sample CSV Log Extract

The following is a representative extract from a CSV detection log file recorded during batch 2 outdoor testing. The extract shows a transition from no detection to confirmed detection as a drone enters the camera field of view.

```
logs > test_2 > Test2_DroneStationary_ONNX_2026-03-20_18-03-28.csv > data
1 timestamp,frame_id,detected,raw_detected,confidence,bbox_x1,bbox_y1,bbox_x2,bbox_y2,bbox_area,fps
2 2026-03-20 18:03:32.749,1,0,1,0.568,102.945,144.347,201.497,179.065,3421.574,0.28
3 2026-03-20 18:03:34.410,2,0,1,0.712,117.246,143.103,214.527,176.416,3240.712,0.6
4 2026-03-20 18:03:34.744,3,1,1,0.752,116.363,143.186,217.873,176.311,3362.544,2.99
5 2026-03-20 18:03:35.020,4,1,1,0.702,116.605,142.591,219.322,175.836,3414.895,3.62
6 2026-03-20 18:03:35.290,5,1,1,0.65,119.095,143.255,223.106,176.264,3433.243,3.7
```

Figure 7: Sample CSV Log Extract Showing Detection Confirmation Sequence

The extract shows the temporal persistence filter in operation. Frames 1 and 2 show raw detections (`raw_detected=1`) with confidence scores of 0.568 and 0.712, but `detected` remains 0 as the persistence counter has not yet reached the required threshold of 3 consecutive frames. From frame 3 onwards `detected=1`, confirming the drone and triggering the servo and buzzer response. The low FPS values in frames 1 and 2 (0.28 and 0.60) reflect the system initialising at script startup before reaching its steady-state throughput of approximately 3.6–3.9 FPS from frame 3 onward.

16 References

- [1] Ultralytics, "YOLO11 Documentation," Ultralytics, 2024. [Online]. Available: <https://docs.ultralytics.com>. [Accessed Throughout. 2026].
- [2] DatasetNinja, "UAV Drone Dataset," DatasetNinja, 2024. [Online]. Available: [Drone Dataset \(UAV\) - Dataset Ninja](#). [Accessed Feb. 2026].
- [3] Roboflow, "Drone Detection Dataset," Roboflow Universe, 2024. [Online]. Available: <https://universe.roboflow.com>. [Accessed Nov. 2025].
- [4] OpenCV, "OpenCV Documentation," OpenCV, 2024. [Online]. Available: [OpenCV: OpenCV modules](#). [Accessed Throughout. 2025 - 2026].
- [5] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, Technical Report TR 95-041, 2006.
- [6] ONNX Runtime, "ONNX Runtime Documentation," Microsoft, 2024. [Online]. Available: <https://onnxruntime.ai/docs>. [Accessed Nov. 2025].
- [7] Raspberry Pi Foundation, "Raspberry Pi Documentation," Raspberry Pi Ltd, 2024. [Online]. Available: <https://www.raspberrypi.com/documentation>.
- [8] Air Sentinel, "Complete Airport Drone Detection Guide," 2025. [Online]. Available: [Airport Drone Detection 2025: How to Choose, Compare & Comply](#). [Accessed Oct. 2025].
- [9] Foreign Policy Research Institute, "Small Drones, Big Problems: Managing the Unmanned Threat to the Homeland," FPRI, 2025. [Online]. Available: [Small Drones, Big Problems: Managing the Unmanned Threat to the Homeland - Foreign Policy Research Institute](#). [Accessed Oct. 2025].
- [10] gpiozero, "gpiozero 2.0.1 Documentation," gpiozero, 2024. [Online]. Available: <https://gpiozero.readthedocs.io>. [Accessed Mar. 2026].
- [11] Random Nerd Tutorials, "Raspberry Pi: MJPEG Streaming Web Server (Picamera2)," [Online]. Available: [Raspberry Pi: MJPEG Streaming Web Server \(Picamera2\) | Random Nerd Tutorials](#). [Accessed Jan. 2026].

- [12] R. Toews, "Deep Learning's Carbon Emissions Problem," Forbes, Jun. 2020. [Online]. Available: [Deep Learning's Carbon Emissions Problem](#) . [Accessed Apr. 2026].
- [13] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [14] M. Lynch, "Machine Learning," Lecture Material, Software and Electronic Engineering, Atlantic Technological University, Galway, 2024.
- [15] AidanM324, "Drone Detection and Response System," GitHub repository, 2026. [Source Code]. Available: https://github.com/AidanM324/Drone_Detection_And_Response_System.git.